

Individual Task 1, Part 1 — What I Did and Why

1. What the assignment actually wants

Part 1 has three moving pieces that all have to line up: pick a real data science job you'd genuinely apply for, find two public datasets that relate to that job, then train and evaluate a model on each one. The point isn't the modelling itself — it's showing that I can connect a business problem (the job) to real data (the datasets) to a defensible modelling choice (why a Decision Tree here, why kNN there), and then read the results like an analyst rather than just reporting numbers.

I used the wiq (Woolworths × Quantum) Data Scientist listing as the target role, since it's directly relevant given I already work at Woolworths. That role leans on two things: pricing/margin analytics and customer personalisation. So I picked one dataset for each half — the Superstore sales data for pricing and profitability, and the UCI Online Retail transactions for customer value — instead of using two datasets that do the same job twice.

2. Model 1 — Decision Tree on the Superstore data

The business question

Given an order line (its sales value, quantity, discount, shipping cost, category, region, etc.), was it profitable or not? This maps onto the pricing/ranging side of the wiq role — understanding what drives margin.

How the code works (model1_decision_tree.py)

- Load the Superstore CSV and build the target: Profit > 0 becomes 1 (profitable), everything else becomes 0. Simple binary label, no fancy binning.
- Pick the features that would realistically be known at order time: Sales, Quantity, Discount, Shipping Cost (numeric) plus Segment, Market, Region, Category, Sub-Category, Order Priority, Ship Mode (categorical).
- Encode the categorical columns with LabelEncoder — a Decision Tree doesn't need one-hot encoding since it just splits on thresholds, so this keeps things simple.
- Split 75/25 into train/test, stratified on the target so the profitable/loss-making ratio stays consistent in both sets.
- Train a DecisionTreeClassifier with max_depth=6 and min_samples_leaf=50 — shallow and with a decent leaf size on purpose, so the tree generalises instead of memorising noisy individual orders. class_weight='balanced' compensates for the fact that most orders are profitable (~74%), so the model doesn't just learn to predict 'profitable' every time.
- Evaluate on the untouched test set with accuracy, precision, recall, F1 and ROC-AUC, plus a confusion matrix, and pull out feature_importances_ to see which columns actually drove the splits.

What came out of it

Discount alone accounted for about 95% of the model's decision-making (feature importance 0.9522), dwarfing every other feature. In plain terms: once you know how heavily an order was discounted, you can predict whether it lost money almost regardless of what was sold or where. That's a genuinely useful, slightly uncomfortable insight for a pricing team — deep discounts are eating margin more than category mix or geography ever does.

3. Model 2 — kNN on the Online Retail data

The business question

Can you tell which customers are high-value before they've spent the money — using only how recently and how often they buy, not how much? This maps onto the personalisation/loyalty side of the role.

How the code works (model2_knn.py)

- Load the raw UK online retail transaction log, drop rows with no CustomerID (can't build customer-level features without one), and keep only positive Quantity and UnitPrice (removes returns and data errors).
- Compute Revenue = Quantity × UnitPrice per line, then group everything by CustomerID into RFM-style features: Recency (days since last order), Frequency (distinct invoices), Monetary (total spend), AvgBasket (average revenue per invoice) and DistinctItems (unique products bought).
- Label the target: top 25% of customers by Monetary spend become 'HighValue' (1), the rest are 0.
- Deliberately exclude Monetary from the predictors — using total spend to predict a label built from total spend would be circular. The model only gets Recency, Frequency, AvgBasket and DistinctItems, so it has to prove those signals alone carry real predictive power.
- Standardise all four features (StandardScaler) — this matters for kNN specifically, since it's a distance-based method and any unscaled feature with a bigger numeric range would silently dominate the distance calculation.
- Split 75/25, stratified, then train KNeighborsClassifier with k=9 and weights='distance', so closer neighbours count more than distant ones when the model votes.
- Same evaluation approach as Model 1: accuracy, precision, recall, F1, ROC-AUC and a confusion matrix.

What came out of it

Even with Monetary held back, the model reached 0.918 ROC-AUC — meaning recency and frequency alone genuinely do distinguish high-value customers from everyone else. That's the more interesting finding for a personalisation team: you don't need someone's full spend history to flag them as high-value, just how often and how recently they've shown up.

4. Reading the two results together

Table 1 (below) summarises both models on their held-out test sets:

Metric	Decision Tree (Superstore)	kNN (Online Retail)
Accuracy	0.896	0.881
Precision	0.958	0.769
Recall	0.899	0.749
F1-score	0.928	0.759
ROC-AUC	0.955	0.918

The two aren't contradictory, they're complementary — one covers the transaction/pricing side, the other the customer/loyalty side. Put together, the story is: don't over-discount at the order level, and invest in re-engaging customers based on how recently and often they buy rather than assuming big spenders are simply loyal. Neither dataset suggests deeper discounting builds loyal high-value customers, which is a nice, checkable claim rather than a vague generalisation.

On evaluation metric choice: both targets are moderately imbalanced (~74/26 and ~75/25), so accuracy on its own would be misleading — a model that always predicts the majority class would already score reasonably well without being useful. That's why precision, recall, F1 and ROC-AUC are all reported rather than just accuracy.

5. Running the scripts in Terminal

This is the step-by-step for running both scripts from scratch on the same Mac setup as the screenshots (.venv + pip).

Step 1 — Open Terminal and go to the project folder

```
cd task1_analysis_code
```

Step 2 — Activate the virtual environment

```
source .venv/bin/activate
```

The prompt should now show (.venv) at the start, same as in the screenshots.

Step 3 — Install the required packages (only needed once)

```
pip install pandas numpy scikit-learn
```

Step 4 — Create the data folder and download the two datasets

```
mkdir -p data
```

```
curl -L 'https://raw.githubusercontent.com/pplonski/datasets-for-start/master/superstore-sales/superstore_dataset2011-2015.csv' -o data/superstore.csv
```

```
curl -L 'https://raw.githubusercontent.com/databricks/Spark-The-Definitive-Guide/master/data/retail-data/all/online-retail-dataset.csv' -o data/online_retail_full.csv
```

-L follows redirects (GitHub raw links sometimes redirect), -o sets the output filename.

Step 5 — Run each script

```
python model1_decision_tree.py
```

```
python model2_knn.py
```

Each one prints its results as JSON straight to the terminal, and also writes a copy to disk (results_model1.json, results_model2.json). Model 2 additionally writes rfm_customers.csv — the per-customer feature table it built along the way.

Common issues

- If python isn't found, try python3 instead — depends on how the venv aliases it.
- A ModuleNotFoundError usually means the venv isn't activated (Step 2 skipped) or Step 3 wasn't run.
- Both scripts expect to be run from task1_analysis_code, since they reference data/... as a relative path — running them from a different folder throws a FileNotFoundError.
- Run 'deactivate' to exit the virtual environment when done.

6. Getting this onto GitHub

The README already tells you the folder structure and how to run the scripts locally. To actually get the repo up and linked in Appendix B:

```
cd task1_analysis_code
git init
git add model1_decision_tree.py model2_knn.py README.md
git commit -m "Initial commit: Decision Tree and kNN models for Task 1"
git branch -M main
git remote add origin https://github.com/Vasanth0701/<your-repo-name>.git
git push -u origin main
```

A few things worth doing before you push:

- Don't commit the actual CSVs (data/superstore.csv, data/online_retail_full.csv) — they're large and easy to re-download from the URLs already in the README. Add a data/ line to a .gitignore file so you don't accidentally commit them.
- Double-check the repo is Public (or that your marker has been given access), since Appendix B links straight to it.
- Once it's pushed, replace the placeholder line in Appendix B of the report ([Update before submitting...]) with the real repo URL before you export the final PDF.
- Optional but a nice touch: commit the two results_model*.json files and the terminal screenshots too, so anyone opening the repo can see the run without re-executing anything.